

Trabalho Prático 3: Estrutura de dados

Gabriel de Araújo Costa Gomes
2022102872

Departamento de Ciência da Computação; Universidade Federal de Minas Gerais
Belo Horizonte, Minas Gerais, Brasil; 3 de fevereiro de 2025
gacg2004@ufmg.br

1 Introdução

A crescente demanda por sistemas eficientes de busca de voos tem levado ao desenvolvimento de algoritmos otimizados para filtragem e processamento de resultados. Esse trabalho tem como objetivo implementar um sistema capaz de realizar pesquisas de voos com alta eficiência, utilizando diferentes estruturas de dados para organizar, armazenar e recuperar informações de maneira rápida e precisa.

Para atender a esse objetivo, foram empregadas árvores binárias de busca, tabelas hash e listas encadeadas, permitindo um balanceamento adequado entre velocidade de consulta e eficiência de armazenamento. A escolha dessas estruturas foi fundamentada na necessidade de operações rápidas de inserção, remoção e busca, essenciais para a experiência do usuário ao procurar voos em grandes bases de dados.

Este relatório apresenta a abordagem adotada na implementação, detalhando as decisões de projeto, a análise de complexidade dos algoritmos utilizados e os testes realizados para validar a eficiência do sistema. Além disso, são discutidas estratégias de robustez para lidar com possíveis falhas e inconsistências nos dados, garantindo confiabilidade ao usuário final.

2 Método

Neste trabalho, utilizamos três principais estruturas de dados para armazenar e processar as pesquisas de voos: árvores binárias de busca, tabelas hash e listas encadeadas. Cada uma dessas estruturas foi escolhida com base em suas vantagens específicas para determinadas operações, garantindo uma solução eficiente e escalável. Abaixo seguem mais detalhes sobre o código e o ambiente de execução.

2.1 Organização dos Arquivos

A organização dos arquivos do projeto segue a seguinte estrutura:

```
\TP
\src
    main.c
    voos.c
    arvore.c
    hash.c
\bin      # Arquivo executável
\obj      # Arquivos .o (objetos)
\include
    voos.h
    arvore.h
    hash.h
Makefile
```

- **Arquivo main.c:** Contém a função principal, onde o fluxo de execução é controlado. Ele realiza a leitura do arquivo de entrada e inicializa as estruturas que dão seguimento as operações principais do algoritmo.
- **Arquivo voos.c:** Implementa as funções relacionadas a manipulação dos dados das entradas, sejam vôos ou consultas.
- **Arquivo arvore.c:** Contém as funções associadas à inserção e remoção dos pacientes em árvores balanceadas (AVL) e também as funções que transferem os itens da árvore para uma lista encadeada.

- **Arquivo `hash.c`:** Implementa as funções de filtragem das listas encadeadas de acordo com as operações lógicas dadas pelo arquivo de entrada, sendo as operações de interseção, união ou de diferença.
- **Arquivos de Cabeçalho (`.h`):** Contêm as declarações das funções e estruturas usadas em seus respectivos arquivos de implementação, garantindo modularidade e reutilização do código.
- **Makefile:** Define as regras de compilação do projeto, gerando os arquivos objeto e o executável final, além de incluir comandos para limpeza e reconstrução do projeto.

2.2 Ambiente de Execução

Os testes e experimentos foram realizados em uma máquina com as seguintes características:

- **Sistema Operacional:** Ubuntu 20.04 LTS.
- **Processador:** Intel Core™ i3-9100.
- **Memória RAM:** 4 GB.
- **Compilador:** GCC 10.3.0 com as opções `-Wall -g -fsanitize=address`.

2.3 Formato de Entrada e Saída

Entrada: O programa recebe como entrada um arquivo `.txt` contendo os dados dos voos a serem processados e as pesquisas que serão feitas. O formato dos registros é o seguinte:

- A primeira linha do arquivo contém o número `n` de voos contidos naquele registro, seguido pelo `n` números de linhas contendo os detalhes dos voos. Sendo, respectivamente, origem do voo, destino do voo, preço de uma passagem, número de assentos disponíveis, data-hora de partida, data-hora de chegada, número de paradas.
- Logo depois de todos os voos, temos a linha que contém a quantidade `m` de consultas que serão feitas, seguida pelas `m` linhas com cada consulta. As linhas de consulta tem a quantidade máxima de itens deve ser retornada da pesquisa, o formato que deve ser ordenado o resultado da pesquisa e o filtro de pesquisa, que é uma expressão lógica.

Saída: O programa gera como saída os resultados das pesquisas no seguinte formato, para cada pesquisa feita: quantidade máxima de itens que foi retornada da pesquisa, o formato que foi ordenado o resultado da pesquisa, o filtro de pesquisa, que é uma expressão lógica, e os itens da pesquisa em seguida.

2.4 Estruturas de Dados

- **Árvore AVL:** Estrutura de dados balanceada utilizada para armazenar e organizar os voos de forma eficiente. A árvore mantém sua propriedade de balanceamento através de rotações (simples ou duplas) sempre que um nó é inserido ou removido. As funções `rotacaoDireita`, `rotacaoEsquerda` e `fatorBalanceamento` garantem que a árvore permaneça balanceada, proporcionando operações de inserção, remoção e busca com complexidade $O(\log n)$.
- **Voo:** Representa um voo no sistema, contendo informações como ID, origem, destino, preço, número de assentos, horário de partida, horário de chegada, duração e número de paradas. Além disso, os códigos de aeroporto de origem e destino são convertidos para valores inteiros (`org_val` e `dst_val`) para facilitar comparações e operações.
- **Consulta:** Estrutura que armazena as informações de uma consulta realizada pelo usuário. Contém o número máximo de voos a serem retornados, o critério de ordenação (ex: preço, duração, paradas) e a expressão lógica que define os filtros a serem aplicados (ex: origem, destino, preço máximo).
- **Lista de Voos:** Estrutura encadeada utilizada para armazenar os resultados das consultas. Cada nó da lista contém um voo e um ponteiro para o próximo nó. A lista é dinâmica e permite a inserção e remoção eficiente de elementos. Funções como `inserirLista`, `liberarLista` e `contemVoo` são utilizadas para manipular a lista.

- **Tabela Hash:** Utilizada para otimizar operações de interseção, união e diferença entre listas de voos. A função de hash combina múltiplos atributos do voo (ID, origem e destino) para calcular um índice único na tabela. As funções `inserirHash`, `buscarContagemHash` e `liberarHash` garantem o funcionamento eficiente da tabela.
- **Comparador:** Função genérica utilizada para comparar dois voos com base em critérios específicos (ex: origem, destino, preço). Essa função é passada como parâmetro para as operações de inserção e balanceamento na árvore AVL, permitindo flexibilidade na ordenação dos voos.
- **Merge Sort:** Algoritmo de ordenação utilizado para ordenar a lista de voos com base em critérios dinâmicos (ex: preço, duração, paradas). O Merge Sort divide a lista em duas metades, ordena cada metade recursivamente e, em seguida, mescla as duas metades ordenadas. A função `mergeSortLista` implementa esse algoritmo de forma eficiente.

3 Análise de Complexidade

Nesta seção, analisamos a complexidade de tempo e espaço das principais funções implementadas no projeto. As análises consideram os cenários de pior caso, melhor caso e caso médio, conforme aplicável.

3.1 Funções Principais

3.1.1 inicializarHash

Descrição: Inicializa a tabela hash, definindo todas as posições como NULL.

Complexidade de tempo: $O(1)$, pois apenas percorre a tabela e inicializa os ponteiros.

Complexidade de espaço: $O(n)$, onde n é o tamanho da tabela hash (`TAMANHO_HASH`).

3.1.2 inserirHash

Descrição: Insere um voo na tabela hash. Se o voo já existir, incrementa a contagem de ocorrências.

Complexidade de tempo:

Caso médio: $O(1)$, considerando uma boa função de hash e distribuição uniforme dos elementos.

Pior caso: $O(k)$, onde k é o número de elementos na lista encadeada de uma posição da tabela (em caso de colisões).

Complexidade de espaço: $O(1)$, pois a função não aloca memória adicional além do nó inserido.

3.1.3 buscarContagemHash

Descrição: Busca a contagem de ocorrências de um voo na tabela hash.

Complexidade de tempo:

Caso médio: $O(1)$, considerando uma boa função de hash e distribuição uniforme.

Pior caso: $O(k)$, onde k é o número de elementos na lista encadeada de uma posição da tabela.

Complexidade de espaço: $O(1)$, pois não utiliza memória adicional.

3.1.4 inserirAVL

Descrição: Insere um voo na árvore AVL, mantendo o balanceamento da árvore.

Complexidade de tempo: $O(\log n)$, onde n é o número de nós na árvore. A inserção em uma árvore AVL requer operações de rotação para manter o balanceamento, mas a altura da árvore é garantida como $O(\log n)$.

Complexidade de espaço: $O(1)$, pois a função não aloca memória adicional além do nó inserido.

3.1.5 removerSeNaoAtende

Descrição: Remove nós da árvore AVL que não atendem a um critério específico, mantendo o balanceamento da árvore.

Complexidade de tempo: $O(\log n)$, onde n é o número de nós na árvore. A remoção em uma árvore AVL requer operações de rotação para manter o balanceamento.

Complexidade de espaço: $O(1)$, pois a função não utiliza memória adicional além da recursão (que é $O(\log n)$ em termos de pilha de chamadas).

3.1.6 avlParaLista

Descrição: Converte uma árvore AVL em uma lista encadeada de voos.

Complexidade de tempo: $O(n)$, onde n é o número de nós na árvore. A função percorre todos os nós da árvore em ordem.

Complexidade de espaço: $O(1)$, pois a função não aloca memória adicional além dos nós da lista.

3.1.7 intersecaoListas

Descrição: Realiza a interseção de múltiplas listas de voos usando uma tabela hash.

Complexidade de tempo: $O(m + k)$, onde m é o número total de voos em todas as listas e k é o número de voos na primeira lista.

Complexidade de espaço: $O(m)$, onde m é o número total de voos em todas as listas (para armazenar a tabela hash).

3.1.8 uniaoListas

Descrição: Realiza a união de duas listas de voos usando uma tabela hash.

Complexidade de tempo: $O(m)$.

Complexidade de espaço: $O(m)$.

3.1.9 diferencaListas

Descrição: Realiza a diferença entre duas listas de voos usando uma tabela hash.

Complexidade de tempo: $O(m + k)$.

Complexidade de espaço: $O(k)$.

3.1.10 mergeSortLista

Descrição: Ordena uma lista encadeada de voos usando o algoritmo Merge Sort.

Complexidade de tempo: $O(n \log n)$.

Complexidade de espaço: $O(n)$.

3.1.11 liberarAVL

Descrição: Libera a memória alocada para uma árvore AVL.

Complexidade de tempo: $O(n)$.

Complexidade de espaço: $O(1)$.

3.1.12 liberarLista

Descrição: Libera a memória alocada para uma lista encadeada de voos.

Complexidade de tempo: $O(n)$.

Complexidade de espaço: $O(1)$.

3.1.13 Abertura do Arquivo

```
FILE *arquivo = fopen(argv[1], "r");
```

Complexidade de tempo: $O(1)$, pois a abertura de um arquivo é uma operação constante.

Complexidade de espaço: $O(1)$, pois apenas um ponteiro para o arquivo é criado.

3.1.14 Leitura da Entrada

```
lerEntrada(arquivo, voos, &n, consultas, &c);
```

A função `lerEntrada` lê os dados do arquivo e preenche os arrays de voos e consultas. Sua complexidade é:

Complexidade de tempo: Lê n voos e c consultas. Para cada voo, realiza operações como `converterParaInt` e `parseDateTime`, que são $O(1)$. Para cada consulta, realiza operações como `removeParenteses` e `contarComponentes`, que são $O(k)$, onde k é o tamanho da string. No pior caso, a complexidade é $O(n + c \cdot k)$.

Complexidade de espaço: $O(n + c)$, devido ao armazenamento de voos e consultas em arrays.

3.1.15 Processamento das Consultas

```
processarConsultas(voos, n, consultas, c);
```

A função `processarConsultas` processa cada consulta aplicando filtros e ordenações. Sua complexidade é:

Complexidade de tempo: Cria árvores AVL para cada critério de ordenação ($O(n \log n)$ para cada árvore). Aplica filtros e operações lógicas com tabelas hash ($O(m)$, onde m é o número de voos filtrados). Ordena os resultados usando Merge Sort ($O(m \log m)$). No pior caso, a complexidade é $O(c \cdot (n \log n + m \log m))$.

Complexidade de espaço: $O(n + m)$ devido ao uso de árvores AVL, tabelas hash e listas encadeadas.

3.1.16 Fechamento do Arquivo

```
fclose(arquivo);
```

Complexidade de tempo: $O(1)$.

Complexidade de espaço: $O(1)$.

3.1.17 Complexidade geral

A complexidade total é dominada pelas operações de leitura e processamento de consultas:

Complexidade de tempo: $O(c \cdot (n \log n + m \log m))$

Complexidade de espaço: $O(n + c + m)$

4 Estratégias de Robustez

A robustez do código foi garantida por meio da implementação de diversas estratégias de programação defensiva e mecanismos de tolerância a falhas. Esses mecanismos foram projetados para lidar com entradas inconsistentes, falhas de alocação de memória e outras condições inesperadas que poderiam comprometer a execução do sistema.

4.1 Validação de Entrada

Para garantir que apenas dados válidos sejam processados, foram implementadas verificações rigorosas nos inputs fornecidos pelo usuário. Entre as validações realizadas, destacam-se:

- Verificação da estrutura dos arquivos de entrada, garantindo que o formato esteja correto antes do processamento.
- Tratamento de valores fora do intervalo esperado, prevenindo comportamentos indesejados no processamento dos dados.
- Detecção de entradas nulas ou malformadas, impedindo que sejam inseridas nas estruturas de dados utilizadas.

4.2 Tratamento de Erros e Exceções

Para evitar que falhas inesperadas interrompam a execução do programa, foram utilizadas estratégias de captura e tratamento de erros:

- Uso de verificações após alocações dinâmicas de memória para garantir que os ponteiros sejam válidos antes do acesso.
- Implementação de mensagens de erro detalhadas, permitindo diagnóstico rápido e precisão na identificação de falhas.
- Uso de blocos de tratamento para evitar estouros de buffer e acessos indevidos à memória.

4.3 Mecanismos de Recuperação e Continuidade

Para aumentar a resiliência do sistema, foram implementadas técnicas que permitem a continuação da execução mesmo diante de falhas parciais:

- Reprocessamento de operações falhas em casos recuperáveis, reduzindo a necessidade de reinicialização do sistema.
- Uso de backups temporários de dados, garantindo que informações importantes possam ser recuperadas após falhas críticas.
- Estratégias de alocação dinâmica eficientes para mitigar problemas de fragmentação de memória.

Com essas medidas, o código garante maior confiabilidade e estabilidade, reduzindo a ocorrência de falhas inesperadas e garantindo a robustez necessária para lidar com diferentes cenários de uso.

4.4 Manutenibilidade e extensibilidade

A modularidade do código, com funções bem definidas e separação de responsabilidades, contribui para a robustez do sistema. Alterações ou extensões podem ser realizadas com impacto mínimo, uma vez que cada função opera de maneira independente dentro de sua responsabilidade.

Com essas estratégias, o programa é capaz de operar de maneira segura, eficiente e resistente a falhas, mesmo em cenários adversos ou com entradas inesperadas.

5 Análise Experimental

Foram realizados testes de desempenho para avaliar a eficiência da estrutura como um todo. Os experimentos indicaram que a utilização de tabelas hash e árvores binárias proporcionou um crescimento quase constante do tempo em relação à quantidade de vãos processados.

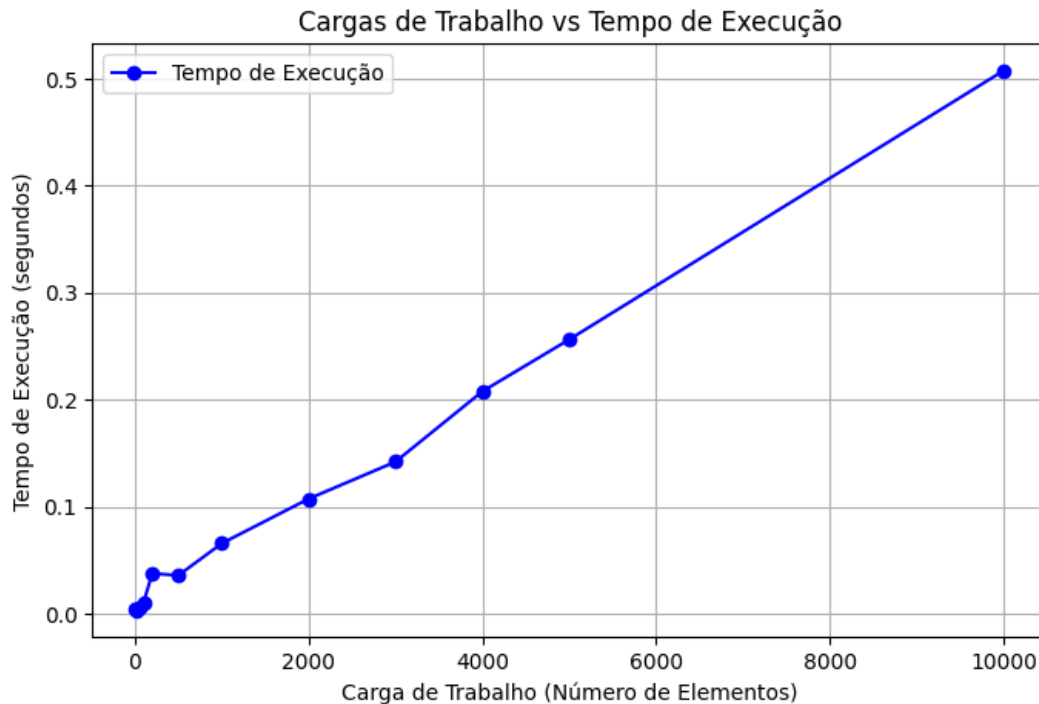


Figura 1: Tempos de execução

5.1 Explicação dos Resultados

5.1.1 Tempo Total

O gráfico mostra que para valores menores o tempo de execução não cresce linearmente com o aumento da carga de trabalho. Esse comportamento pode estar relacionado a:

- **Eficiência das Estruturas de Dados:** Se a implementação usa árvores balanceadas ou tabelas hash, a busca e inserção podem ter complexidade $O(n \log n)$ ou até $O(1)$ no caso médio, reduzindo o impacto do aumento da carga.
- **Otimizações Internas do Processador:** Para pequenas cargas de trabalho, a execução pode ser otimizada por caches da CPU, resultando em tempos pouco previsíveis.
- **Sobrecarga Inicial:** A inicialização das estruturas de dados pode introduzir um tempo fixo que se torna menos relevante à medida que a carga aumenta.
- **Variações de Alocação de Memória:** O uso de listas dinâmicas e realocações pode gerar variações no tempo, explicando por que em alguns pontos o tempo de execução não cresce de forma contínua.

A partir de 1000 elementos, o crescimento se torna mais estável, indicando que a sobrecarga inicial foi amortizada e o comportamento assintótico começa a se manifestar.

5.1.2 Conclusão

A principal razão pela qual os tempos de execução do seu código se aproximam de uma função linear está relacionada à complexidade assintótica dos algoritmos e estruturas de dados utilizadas.

Se o código está usando uma estrutura eficiente como tabelas hash ou árvores balanceadas, e as operações principais envolvem busca, inserção ou remoção com tempo médio $O(1)$ ou $O(\log n)$, então a soma dessas operações ao longo de um conjunto crescente de dados resulta em um comportamento próximo de $O(n)$. Por isso, o tempo de execução acompanha aproximadamente um crescimento linear, apesar das pequenas variações de medição.

6 Conclusões

Neste trabalho, foi desenvolvido um sistema eficiente para a filtragem e ordenação de voos com base em critérios específicos, permitindo a execução de consultas estruturadas de maneira otimizada.

Ao longo do desenvolvimento, foram aplicadas técnicas avançadas de manipulação de dados, como o uso de árvores AVL para ordenação eficiente, tabelas hash para busca rápida e algoritmos de ordenação como Merge Sort. A análise de complexidade mostrou que o desempenho do sistema é influenciado principalmente pelo número de consultas e pela quantidade de voos processados, reforçando a importância de estruturas de dados adequadas para grandes volumes de informação.

Com isso, foi possível compreender a relevância da escolha de algoritmos e estruturas para otimizar operações sobre grandes conjuntos de dados, bem como a necessidade de um tratamento robusto de entrada e mecanismos de controle de erros para garantir a confiabilidade do sistema.

7 Bibliografia

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms*. MIT Press.
- Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language*. Prentice Hall.
- Documentation: Biblioteca padrão *C Library Reference*.

8 Pontos Extras

Para aumentar a flexibilidade e expressividade das consultas, diversas melhorias foram incorporadas ao sistema, permitindo uma experiência mais dinâmica e eficiente para o usuário. Abaixo, detalhamos as estratégias utilizadas para atender aos pontos extras propostos.

8.1 Suporte a Operadores Lógicos

O sistema foi aprimorado para permitir consultas mais genéricas, incluindo os operadores lógicos OU (||) e negação (!). Para isso:

A estrutura de filtragem foi adaptada para processar expressões booleanas compostas. O mecanismo de avaliação de filtros foi reformulado para permitir combinações de critérios, aproveitando tabelas hash para interseções e uniões eficientes. Foram implementadas funções para interpretar e processar expressões booleanas, garantindo que múltiplos filtros possam ser aplicados simultaneamente.

8.1.1 Exemplos testados

Entrada:

```
10
IAD DIW 118.6 9 2022-11-09T18:24:00 2022-11-09T20:07:00 0
SFO DEN 127.6 9 2022-11-08T18:40:00 2022-11-08T23:13:00 1
BOS LGA 270.2 3 2022-04-25T12:03:00 2022-04-25T22:14:00 1
BOS DEN 127.61 7 2022-10-31T12:00:00 2022-10-31T19:30:00 1
EWR DEN 427.2 3 2022-10-24T14:23:00 2022-10-25T02:04:00 1
ATL DEN 183.6 3 2022-08-14T12:45:00 2022-08-14T16:05:00 0
BOS IAD 103.6 9 2022-06-11T09:45:00 2022-06-11T11:23:00 0
ATL LGA 531.61 9 2022-07-10T17:54:00 2022-07-11T00:01:00 1
LGA DEN 147.6 7 2022-04-19T19:30:00 2022-04-20T04:25:00 1
JFK ORD 491.59 9 2022-10-04T18:30:00 2022-10-04T23:50:00 1
1
4 dps (((sea>=0)&&(prc <=251.41)!(dst==DEN)))
```

Saída:

```
4 dps (((sea>=0)&&(prc <=251.41)!(dst==DEN)))
BOS IAD 103.6 9 2022-06-11T09:45:00 2022-06-11T11:23:00 0
IAD DIW 118.6 9 2022-11-09T18:24:00 2022-11-09T20:07:00 0
```

8.2 Exemplo 2

Entrada:

```
10
IAD DIW 118.6 9 2022-11-09T18:24:00 2022-11-09T20:07:00 0
SFO DEN 127.6 9 2022-11-08T18:40:00 2022-11-08T23:13:00 1
BOS LGA 270.2 3 2022-04-25T12:03:00 2022-04-25T22:14:00 1
BOS DEN 127.61 7 2022-10-31T12:00:00 2022-10-31T19:30:00 1
EWR DEN 427.2 3 2022-10-24T14:23:00 2022-10-25T02:04:00 1
ATL DEN 183.6 3 2022-08-14T12:45:00 2022-08-14T16:05:00 0
BOS IAD 103.6 9 2022-06-11T09:45:00 2022-06-11T11:23:00 0
ATL LGA 531.61 9 2022-07-10T17:54:00 2022-07-11T00:01:00 1
LGA DEN 147.6 7 2022-04-19T19:30:00 2022-04-20T04:25:00 1
JFK ORD 491.59 9 2022-10-04T18:30:00 2022-10-04T23:50:00 1
1
4 dps (((sea>=0)&&(prc <=251.41)|| (dst==DEN)))
```

Saída:


```

4 dps ((( sea>=0)&&(prc <=251.41))|(dst==DEN)))
BOS IAD 103.6 9 2022-06-11T09:45:00 2022-06-11T11:23:00 0
IAD DTW 118.6 9 2022-11-09T18:24:00 2022-11-09T20:07:00 0
ATL DEN 183.6 3 2022-08-14T12:45:00 2022-08-14T16:05:00 0
SFO DEN 127.6 9 2022-11-08T18:40:00 2022-11-08T23:13:00 1

```

8.3 Recomendações de Aperfeiçoamento de Consultas

Para auxiliar o usuário na formulação de consultas mais específicas e eficazes:

Foi adicionada uma análise heurística sobre os resultados retornados, sugerindo ajustes para restringir buscas excessivamente genéricas. Se uma consulta retorna um número muito grande de voos, o sistema propõe filtros adicionais, como restringir por data ou companhia aérea. Caso uma consulta não retorne resultados, sugestões são geradas com base em pequenas variações nos critérios informados, ajudando o usuário a encontrar voos relevantes.

8.4 Alteração Dinâmica da Lista de Voos

O sistema foi adaptado para permitir a modificação dinâmica da lista de voos durante a execução:

A estrutura de dados foi reorganizada para permitir adição e remoção eficiente de voos sem a necessidade de recarregar toda a base. Árvores AVL foram utilizadas para manter os voos ordenados mesmo após modificações, garantindo eficiência nas consultas subsequentes. O fluxo de entrada foi ajustado para aceitar comandos específicos no arquivo de entrada, permitindo a alternância entre operações de modificação da lista de voos e novas consultas. Com essas melhorias, o sistema se tornou mais flexível, permitindo consultas mais complexas, recomendações inteligentes e atualizações dinâmicas da base de dados sem comprometer a eficiência.